CLASS 9b

SUPERVISED FINE-TUNING (SFT)

From Autocompleter to Assistant

`SmolLM-135M + Financial Q&A + Unsloth`

Where We Are

Base Model (next-token predictor)

CPT (Continued Pre-Training) [DONE - Class 9a]
Taught domain language

SFT (Supervised Fine-Tuning) **[TODAY]**
Teaches instruction following

RLHF / DPO [Next session]
Teaches human preferences

RLVR [Next session]
Teaches reasoning

The Problem SFT Solves

Prompt: "What is EBITDA?"

Base model / After CPT:

"What is EBITDA? EBITDA is a measure of a company's operating performance. EBITDA stands for Earnings Before Interest, Taxes, Depreciation, and Amortization. EBITDA is used by analysts to compare..."

→ CONTINUES. Doesn't know when to stop.
→ Doesn't know this is a QUESTION.

After SFT:

"EBITDA stands for Earnings Before Interest, Taxes, Depreciation, and Amortization. It measures a company's operating profitability by excluding non-operating expenses. Investors use it to compare companies across different tax and capital structures."

→ Concise. Answers the question. Stops.

SFT teaches: when you see an instruction, produce a response.
When you're done responding, stop.

What IS SFT?

SFT = train the model on (instruction, response) pairs

Same training loop as CPT: Forward pass → loss → backward pass → update

The ONLY difference is:

1. THE DATA FORMAT

```
CPT: "raw text raw text raw text..."
SFT: "### Instruction: Q ### Response: A"
```

2. WHERE THE LOSS IS COMPUTED

```
CPT: loss on ALL tokens
SFT: loss ONLY on response tokens
```

Why Does SFT Work with So Few Examples?

The "Superficial Alignment Hypothesis" (Lima paper, 2023):

"A model's knowledge and capabilities are learned almost entirely during pre-training. Alignment teaches it the FORMAT of interaction."

Pre-training: Trillions of tokens → knowledge + capability

SFT: Thousands of examples → behavior pattern

The model ALREADY knows how to answer questions.
It saw millions of Q&A pages during pre-training.
SFT just teaches it: "when you see THIS template, activate THAT existing capability."

It's not learning new skills. It's learning the UI.

That's why 1,000–10,000 examples suffice. You're teaching the interface, not the intelligence.

The Alpaca Format

Data format (JSON):

```
{
  "instruction": "What is EBITDA?",
  "input": "",
  "output": "EBITDA stands for Earnings
            Before Interest, Taxes..."
}
```

Turned into a training prompt:

```
Below is an instruction that describes
a task. Write a response that
appropriately completes the request.

### Instruction:
What is EBITDA?

### Response:
EBITDA stands for Earnings...{EOS}
```

The EOS token at the end is critical — that's how the model learns when to stop.

Three fields: instruction, input, output. Named after Stanford's Alpaca project.

Chat Template Format (Alternative)

```
{"messages": [
  {"role": "system", "content": "You are a financial analyst."},
  {"role": "user", "content": "What is EBITDA?"},
  {"role": "assistant", "content": "EBITDA stands for..."}
]}
```

Each model has its OWN special tokens:

SmolLM: `<|im_start|>user\n...<|im_end|>`

Llama 3: `<|start_header_id|>user<|end_header_id|>...<|eot_id|>`

Qwen: `<|im_start|>user\n...<|im_end|>`

NEVER format these manually.

Use: `tokenizer.apply_chat_template(messages)`

Loss Masking — The Key Difference from CPT

Training text: "### Instruction: What is EBITDA? ### Response: EBITDA stands for..."

CPT loss (all tokens):

[###] [Inst] [:] [What] [is] [EB] [IT] [DA] [?] [###] [Resp] [:] [EB] [IT]...

LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS LOSS

SFT loss (response only):

[###] [Inst] [:] [What] [is] [EB] [IT] [DA] [?] [###] [Resp] [:] [EB] [IT]...

-100 -100 -100 -100 -100 -100 -100 -100 -100 -100 -100 -100 LOSS LOSS

-100 = "ignore this token in loss computation"
PyTorch's cross-entropy skips any label that's -100.

Loss Masking — The Implementation

```
# What the trainer does internally:

text = "### Instruction:\nWhat is revenue?\n### Response:\nRevenue is...<EOS>"

input_ids = tokenizer(text).input_ids
labels = input_ids.clone()

# Find where "### Response:" ends
response_start = find_response_start(input_ids)

# Mask everything BEFORE the response
labels[:response_start] = -100

# Loss only computed where labels != -100
loss = cross_entropy(logits, labels)
```

Unsloth's SFTTrainer with the Alpaca template does this automatically.

Packing with Multiple SFT Examples

Without packing (wasteful):

```
Seq 1: [Q&A pair 1] [PAD] [PAD] [PAD] [PAD] [PAD]
Seq 2: [Q&A pair 2] [PAD] [PAD] [PAD] [PAD]
```

With packing (efficient):

```
Seq 1: [Q&A pair 1] [EOS] [Q&A pair 2] [EOS] [Q&A pair 3]
```

But doesn't pair 2 attend to pair 1?

No — the trainer uses ATTENTION MASKING:

- Each packed example only attends to itself
- EOS tokens act as boundaries
- No cross-contamination. Handled automatically by Unsloth and TRL.

The EOS Token — Teaching the Model to Stop

Without EOS training:

```
User: "What is revenue?"

"Revenue is the total income generated from
business operations. Revenue is also known as
sales or turnover. Revenue can be calculated..."
```

→ Never stops.

With EOS training:

```
User: "What is revenue?"

"Revenue is the total income a company generates
from its business operations." [EOS]
```

→ Stops when done.

Every training example ends with EOS.
The model learns: complete response, then STOP.

LoRA Config — CPT vs SFT

	CPT	SFT
rank (r)	32	16
target_modules	all linear + embed_tokens + lm_head	all linear (no embed_tokens no lm_head)
lora_alpha	32	16
learning_rate	2e-4	2e-4
embedding_lr	2e-5	not needed

Why lower rank? SFT = behavior change (fewer directions of change)
Why no embed/lm_head? Vocabulary is fine, behavior needs to change

The Dataset — virattt/financial-qa-10K

7,000 financial Q&A pairs from SEC 10-K filings

```
{
  "question": "What was the total revenue for FY2023?",
  "context": "The company reported total revenues of
              $53.8 billion for the fiscal year ended...",
  "answer": "The total revenue for FY2023 was $53.8 billion."
}
```

We convert to Alpaca: instruction=question, input=context, output=answer

Data quality >> quantity:

- GPT-3.5 was SFT'd on ~12,000 examples
- Lima paper: 1,000 good > 50,000 noisy
- 7,000 high-quality pairs is plenty

The Plan

1. Load SmolLM-135M, ask it a question → autocompletes
2. Load financial Q&A dataset, convert to Alpaca format
3. SFT with Unsloth + LoRA (rank 16)
4. Ask the same questions → it answers!
5. Try edge cases: out-of-domain, no template
6. Merge adapter into base model for deployment

What Happens During SFT Inference

1. User provides: "### Instruction:\nWhat is EBITDA?\n### Response:\n"
2. Model sees the familiar template structure
3. Model generates tokens one by one
4. Model outputs EOS token
5. Generation stops

The template is the INTERFACE between human and model.
The model learned: "when I see ### Response:, produce an answer."

Merging Adapters for Deployment

After training you have: Base model + LoRA adapter

1. Keep separate

`Load base + adapter at runtime`

Pro: swap adapters. Con: slightly more complex.

2. Merge

`W_new = W + (alpha/r) * B * A`

Pro: single model, zero overhead. Con: can't unmix.

3. Push to Hub

`Upload to HuggingFace`

Pro: share with the world. One line to use.

```
model.merge_and_unload()                          # merge
model.push_to_hub("your-name/financial-qa-smollm") # share
```

The Full Pipeline In Practice

1. Base model (SmolLM, Llama, Qwen)
2. CPT on domain text (raw SEC filings) → Learns financial language
3. SFT on instruction-response pairs (financial Q&A) → Learns to answer
4. (Optional) DPO/RLHF on preference data → Learns human preferences
5. Merge all adapters
6. Deploy with RAG for real-time knowledge

CPT without SFT = sounds smart, can't help.

SFT without CPT = follows instructions, limited domain knowledge.

CPT + SFT = follows instructions WITH domain expertise.

PART 3

THE REAL SKILL

Creating Instruction Data from Raw Text

Why You Can't Just Use Public Datasets

Public SFT datasets (Alpaca, ShareGPT, UltraChat):

Generic, stale, wrong domain, quality varies

What you actually need:

- YOUR domain's questions
- YOUR format requirements
- YOUR accuracy standards
- Updated when your domain changes

Solution: Use an LLM to GENERATE instruction data from YOUR raw text.

The Synthetic Data Pipeline

```
Raw domain text (SEC filings, docs, wikis)
        |
    Chunk into passages (500 words)
        |
    For each chunk, ask an LLM to generate:
      - Q&A pairs (factual, analytical, synthesis)
      - Summaries
      - Unanswerable questions (teaches refusal!)
        |
    Quality filter (remove bad examples)
        |
    Format to Alpaca template + EOS
        |
    Train with SFT
```

Strong model runs once for data gen. Small model runs cheaply in production forever.

Prompt Engineering for Data Generation

Bad prompt:

```
"Generate questions from this text."
```

→ Shallow, repetitive, yes/no questions

Good prompt:

```
"Generate 4 diverse Q&A pairs:
  1. Factual (who/what/how much)
  2. Analytical (why/compare)
  3. Synthesis (summarize)
  4. Unanswerable (context lacks)
  Answers cite specific numbers."
```

Prompt quality → Data quality → Model quality

Quality Control — Good vs Bad Examples

GOOD:

```
Q: "What was the year-over-year change in operating expenses?"
A: "Operating expenses increased $1.1B, a 15.5% YoY increase."
```

[specific] [accurate] [cites numbers] [concise]

BAD:

- Answer contradicts the context (hallucinated numbers)
- Answer is just the context copy-pasted
- Question is unanswerable from the context
- Answer is 3 paragraphs for a simple fact
- Missing EOS token
- Inconsistent format

Always sample 50-100 examples and read them manually.
1 hour auditing saves 10 hours debugging a broken model.

PART 4

FAILURE MODES AND HONEST TRADEOFFS

Failure Mode 1 — Sycophancy

User: "Revenue was $100 billion, right?" Context says: Revenue was $53 billion.

Sycophantic:

"Yes, you're right! Revenue was ~$100 billion."

Correct:

"Actually, revenue was $53 billion, not $100 billion."

WHY: Training data has too many "agreeable" answers.
FIX: Include examples where the model corrects the user.

Failure Mode 2 — Hallucination Amplification

Q: "What was the employee count?" Context: [only revenue/income data, NO employee info]

Bad model:

"The company had approximately 47,000 employees."
(completely made up!)

Good model:

"The provided context does not contain information about employee count."

WHY: SFT teaches confident answering. Not "I don't know."
FIX: Include "unanswerable" examples in training data.

Critical for finance where making up numbers has legal consequences.

Failure Mode 3 — Format Overfitting

With template:

```
"### Instruction:\nWhat is EBITDA?\n### Response:
\n"
```

→ "EBITDA stands for..." [works!]

Without template:

```
"What is EBITDA?"
```

→ random autocomplete [broken!]

WHY: Model learned ONE exact template, not "answering questions."
FIX: Train on MULTIPLE prompt formats. Mix Alpaca + raw questions + chat template.

The Alignment Tax

```
Before SFT: Decent at text completion, translation, code, creative writing.
After SFT on financial Q&A: Great at financial Q&A.
            Worse at code. Worse at creative writing.

The model "forgets" behaviors not in the SFT dataset.
```

Mitigation: Mix domain-specific + general instruction data.
80% domain, 20% general. Same principle as CPT mixing.